

# Report: Analysis of NLSCYA data

## Objectives

- Does relaxed lasso in the meta model results in a sparser selection of domains while retaining a reasonably high predictive performance compared to using regular lasso?

## Summary & Conclusions

- At lambda.min, the relaxed lasso parameter gamma is largely 1 for both MVS default and MVS residual models configurations. In this setting, no adjustment to coefficient shrinkage is applied, resulting in the same results compared to using the regular lasso.
- At lambda.1se, the gamma is largely **below** 1 for both configurations, resulting in a sparser set of domains with coefficient shrinkage reduced through the relaxed lasso refitting.
- Using lambda.1se results in a sparser model but at considerable costs in prediction accuracy. The results show a drop of mean predictive R squared by approximately 0.09, from 0.20 for MVS default and 0.21 for MVS residual models, to 0.11 and 0.12, respectively.
- Under lambda.1se, the sparser model for the MVS linear configuration returns a non-zero for one domain, namely mother characteristics. In the MVS residual model configuration, three domains are selected for the linear base model, and one for the random forest base model fitted on the residualized outcome.
- MVS residual models shows a slightly better prediction performance than MVS default for both lambda.min and lambda.1se settings, but the difference is small.

## Data

The data is a sample of the Children of the National Longitudinal Survey of Youth 1979, based on a pre-selection by Deming (2009).

For the analysis we selected cases based on available data, resulting in a sample of 703 children. The outcome variable is a combined cognitive test score.

The covariate group labels are:

- Child characteristics ( $n\_covariates = 8$ ),
- Family characteristics ( $n\_covariates = 4$ ),
- Mother characteristics ( $n\_covariates = 5$ ),
- Pre- and postnatal characteristics ( $n\_covariates = 2$ ).

## Cross-validation procedure

For evaluating predictive performance, 10-fold cross-validation was used. Because the data included siblings, mother's ID was used to ensure that the children with the same mother were assigned to the same fold. The size of each fold was around 70.

## Results

### Using lambda.min setting

**Table 1**

*Predictive performance*

|            | MVS Linear | MVS Residual Models |
|------------|------------|---------------------|
| MSE (SD)   | 482 (65)   | 476 (64)            |
| Test $R^2$ | 0.20       | 0.21                |

*Note.* The variance of the outcome in the complete data is 605. Predictive  $R^2$  was calculated as  $1 - mse / var(y\_test)$ .

**Table 2**

*Medians of standardized coefficients*

|                  | MVS Linear | MVS Residual Models |               |
|------------------|------------|---------------------|---------------|
|                  |            | Linear              | Random Forest |
| Child            | 0.15       | 0.08                | 0.08          |
| Mother           | 0.24       | 0.20                | 0.04          |
| Family           | 0.10       | 0.10                | 0.11          |
| Pre- & Postnatal | 0.13       | 0.10                | <b>0</b>      |

### Using lambda.1se setting

**Table 1**

*Predictive performance*

|                     | MVS Linear | MVS Residual Models |
|---------------------|------------|---------------------|
| MSE (SD)            | 542 (60)   | 531 (58)            |
| Test R <sup>2</sup> | 0.11       | 0.12                |

*Note.* The variance of the outcome in the complete data is 605. Predictive R<sup>2</sup> was calculated as  $1 - \text{mse} / \text{var}(y_{\text{test}})$ .

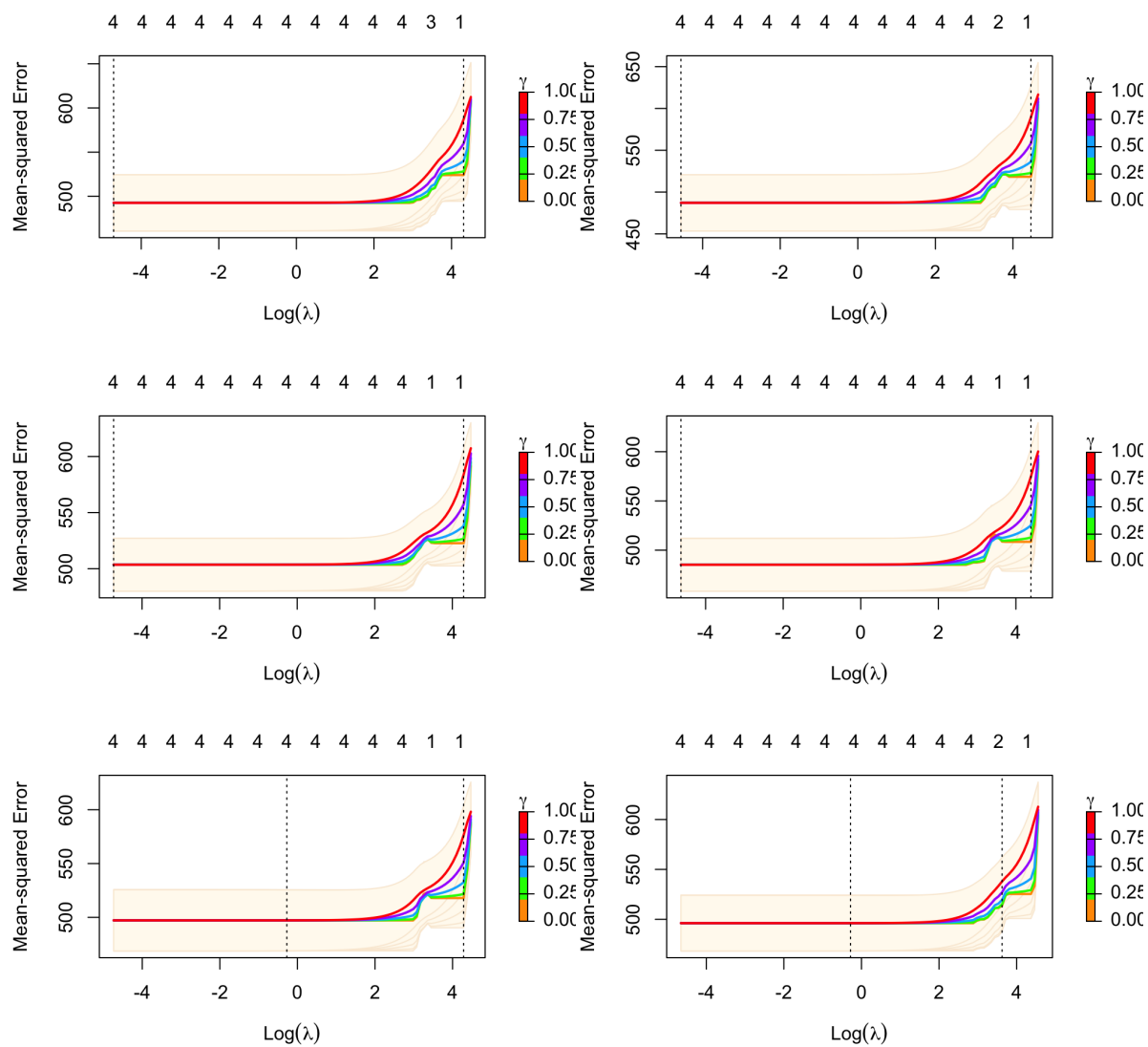
**Table 2**

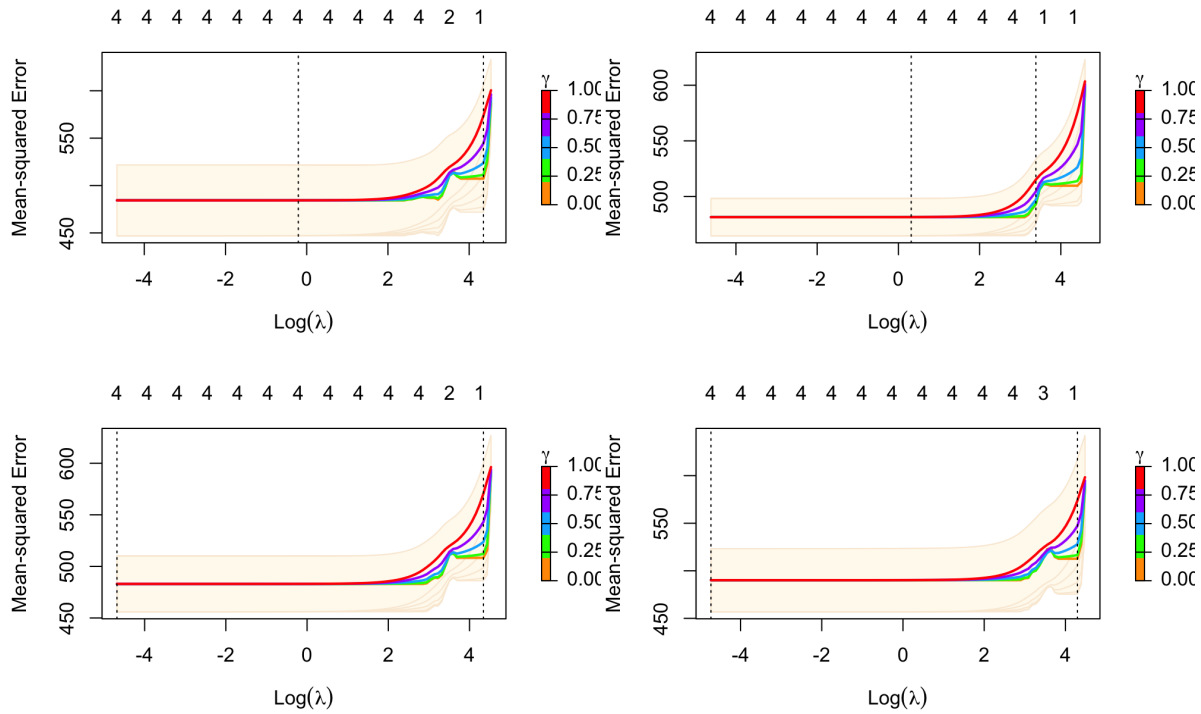
*Medians of standardized coefficients*

|                  | MVS Linear | MVS Residual Models |               |
|------------------|------------|---------------------|---------------|
|                  |            | Linear              | Random Forest |
| Child            | 0          | 0.06                | 0.02          |
| Mother           | 0.39       | 0.25                | 0             |
| Family           | 0          | 0                   | 0             |
| Pre- & Postnatal | 0          | 0.07                | <b>0</b>      |

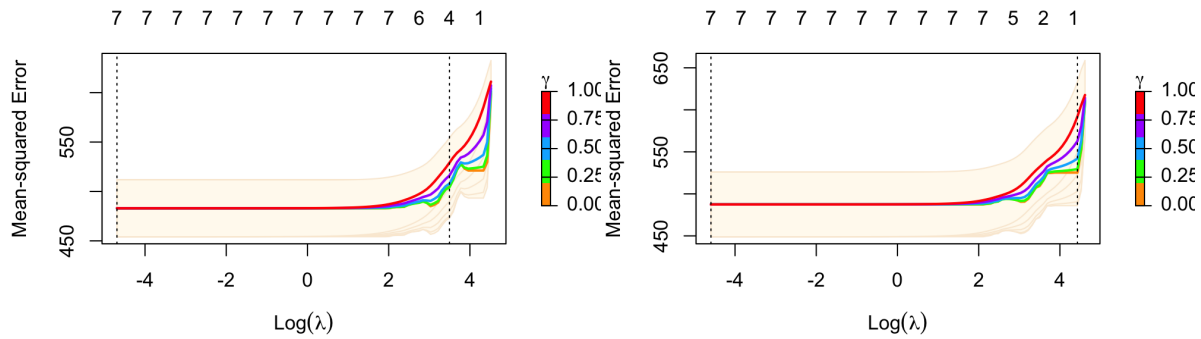
## Plots of lambdas

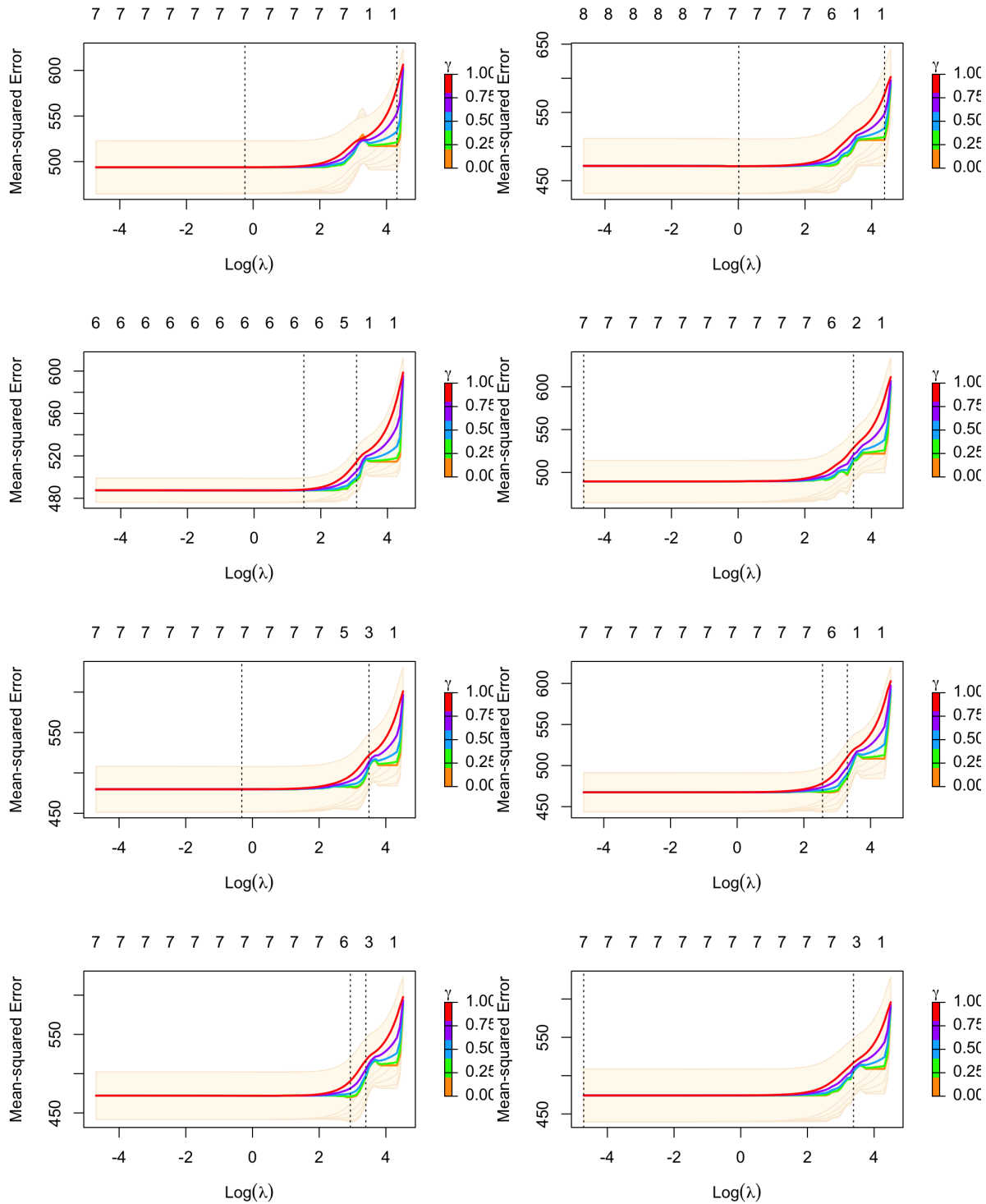
### MVS default





### MVS residual models

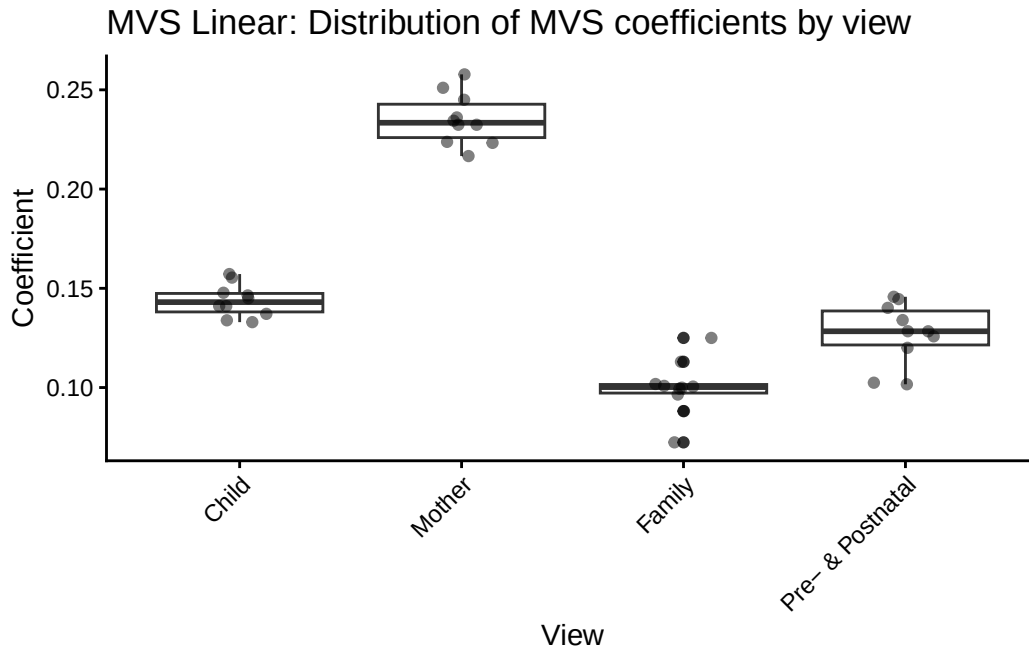


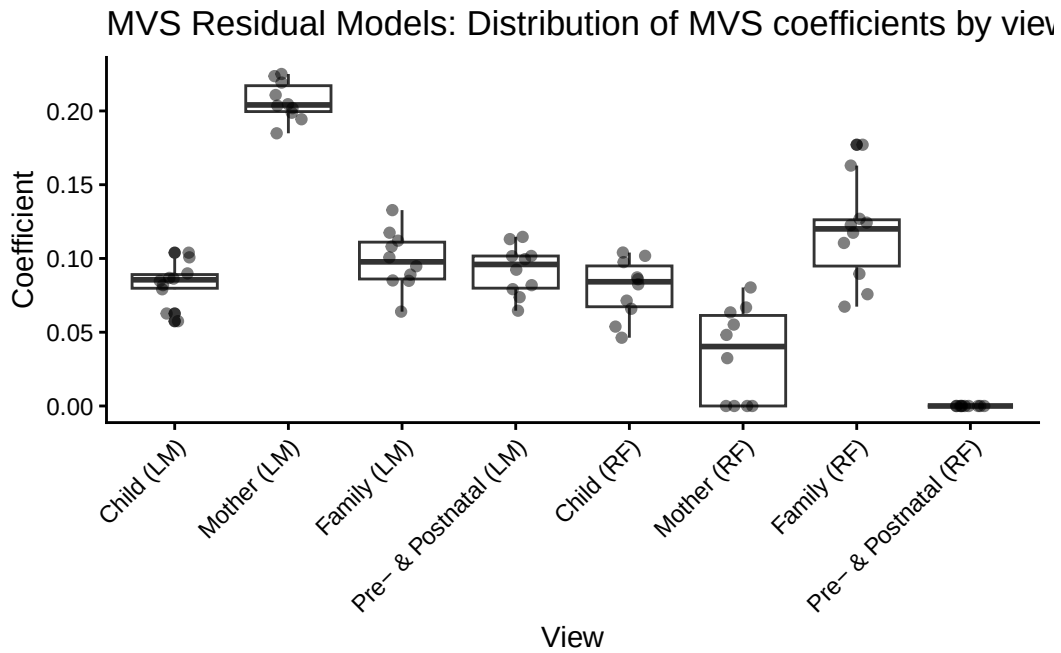


## Plots of coefficient distributions

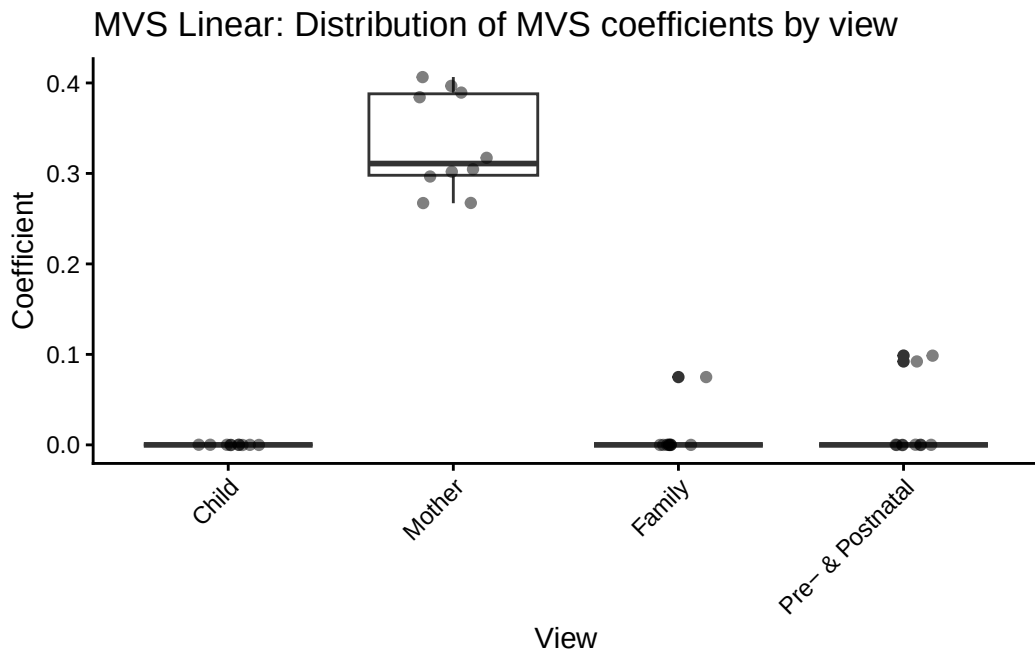
- The plots show the coefficient distributions obtained over the different fold combinations.
- In MVS residual models, views 1 to 4 correspond to linear models fitted to the covariate groups, and views 5 to 8 correspond to random forest models fitted to the same covariate groups but on the residualized outcome.

## Plots for lambda.min

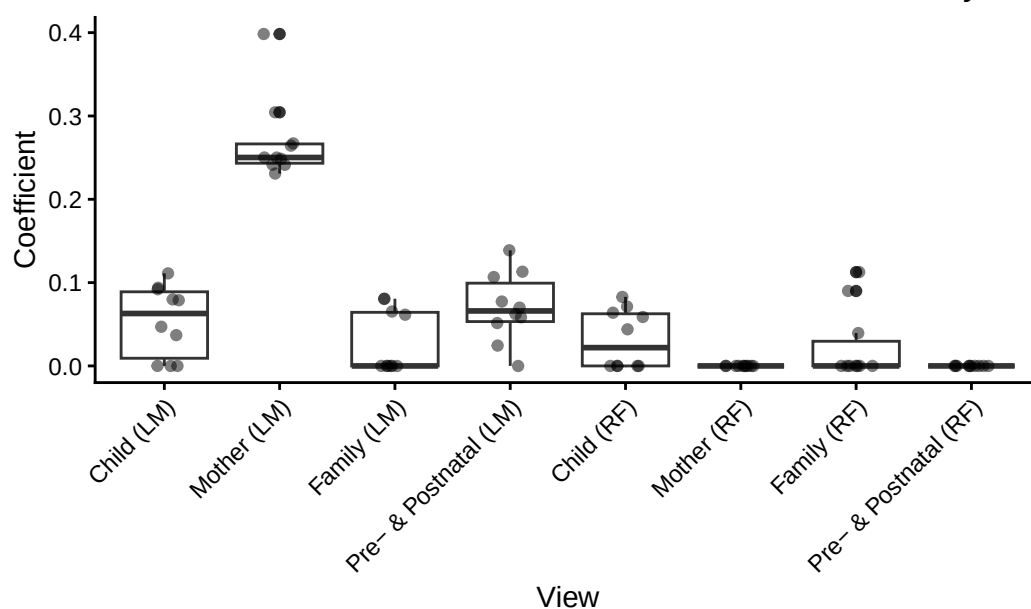




### Plots for lambda1SE



MVS Residual Models: Distribution of MVS coefficients by view



## Variable names and group labels

|    | Var.names         | Group                  |
|----|-------------------|------------------------|
| 1  | Race_Child        | child chr              |
| 2  | Sex_Child         | child chr              |
| 3  | BirthWeight       | child chr              |
| 4  | BirthOrder        | child chr              |
| 5  | BornEarlyorLate   | child chr              |
| 6  | BornOnTime        | child chr              |
| 7  | RepeatNone98      | child chr              |
| 8  | Illness_1stYr     | child chr              |
| 9  | PermInc           | family chr             |
| 10 | FamilySize79      | family chr             |
| 11 | Grandmother86     | family chr             |
| 12 | HighGrade_GDad79  | family chr             |
| 13 | AgeAFQT           | mother chr             |
| 14 | MomDropout        | mother chr             |
| 15 | Height_Mom81      | mother chr             |
| 16 | MothED            | mother chr             |
| 17 | Age_Moth_Birth    | mother chr             |
| 18 | Freq_Alc_BefBirth | Pre- and postnatal chr |
| 19 | Breastfed         | Pre- and postnatal chr |

## Model fitting syntax & helper functions

### Fitting MVS configurations

```
# Libraries
library(glmnet)
library(randomForest)
library(mvs)

# Helper function to extract results
extract_results <- function(model, y_train = y_train, newx = X_test, newy = y_test) {
  pred <- predict(model, newx = newx)
  test_mse <- mean((pred - newy)^2)
  test_r2 <- 1 - test_mse / var(newy)
  coefs <- coef(model)$`Level 2`[[1]]
  cv_variances <- apply(model$`Level 1`$CVs, 2, var)
  cv_sds <- sqrt(cv_variances)
  std_coefs <- coefs[-1,] * (cv_sds / sd(y_train))

  list(
    model = model,
    coefficients = coefs,
    std_coefficients = std_coefs,
    test_MSE = test_mse,
    test_R2 = test_r2,
    cv_variances = cv_variances
  )
}

# Helper function to extract results
extract_results_lambda1se <- function(model, y_train = y_train, newx = X_test, newy = y_test) {
  pred <- predict(model, newx = newx, cvlambda = "lambda.1se")
  test_mse <- mean((pred - newy)^2)
  test_r2 <- 1 - test_mse / var(newy)
  coefs <- coef(model, cvlambda = "lambda.1se")$`Level 2`[[1]]
  cv_variances <- apply(model$`Level 1`$CVs, 2, var)
```

```

cv_sds <- sqrt(cv_variances)
std_coefs <- coefs[-1,] * (cv_sds / sd(y_train))

list(
  model = model,
  coefficients = coefs,
  std_coefficients = std_coefs,
  test_MSE = test_mse,
  test_R2 = test_r2,
  cv_variances = cv_variances
)
}

# Helper function to create cv folds based on group IDs
make_group_cv_folds <- function(data, group_var, k = 10, seed = 123) {
  set.seed(seed)

  # Get unique group IDs
  groups <- unique(data[[group_var]])

  # Shuffle group IDs
  groups <- sample(groups)

  # Assign each group to a fold
  fold_ids <- rep(seq_len(k), length.out = length(groups))
  group_fold_map <- data.frame(
    group = groups,
    fold = fold_ids
  )

  # Create folds
  cv_fold <- group_fold_map$fold[
    match(data[[group_var]], group_fold_map$group)
  ]

  return(cv_fold)
}

# Load data
subset_NLSCYA_cc <- readRDS('NLSCYA_data_cc.rds')

```

```

names(subset_NLSCYA_cc)

# MVS application to NHANES data ----

# Set seed
set.seed(42)

# Create a vector with CV folds
fold_inx <- make_group_cv_folds(subset_NLSCYA_cc, "MotherID")

# Exclude child- and mother ID
deming_data_wthout_IDs <- subset_NLSCYA_cc[,!names(subset_NLSCYA_cc) %in% c("MotherID", "Chi

# Define view indices based on source, taking into account dummies for factors
view_ids <- c(
  rep(1, 16), # Child characteristics
  rep(2, 6), # Mother characteristics
  rep(3, 5), # Family characteristics
  rep(4, 3) # Pre- and postnatal conditions
)

# Create list objects to collect extracted results
res_mvs_default <- list()
res_mvs_rf <- list()
res_mvs_repeated <- list()
res_mvs_resid <- list()

for(i in 1:10) {

  # Split into training and test data

  X_train <- model.matrix( ~ . - 1, data = deming_data_wthout_IDs[fold_inx != i, -20])
  y_train <- deming_data_wthout_IDs[fold_inx != i, 20]
  X_test <- model.matrix( ~ . - 1, data = deming_data_wthout_IDs[fold_inx == i, -20])
  y_test <- deming_data_wthout_IDs[fold_inx == i, 20]

  # Fit MVS models ----

  # Fit MVS default
  mvs_default <- MVS(
    x = X_train,
    y = y_train,

```

```

    views = view_ids,
    progress = FALSE,
    family = "gaussian",
    relax = c(FALSE, TRUE)
  )

# Compute results
# res_mvs_default[[i]] <- extract_results(mvs_default, y_train = y_train, newx = X_test, newy = y_test)
res_mvs_default[[i]] <- extract_results_lambda(mvs_default, y_train = y_train, newx = X_test, newy = y_test)

## RF MVS (four views and groups, random forest baselearners, lasso metalearner);
type <- list(level1 = "RF", meta = "StaPLR")
mvs_rf <- MVS(
  x = X_train,
  y = y_train,
  views = view_ids,
  type = type,
  progress = FALSE,
  family = "gaussian",
  relax = c(FALSE, TRUE)
)

# Compute results
res_mvs_rf[[i]] <- extract_results(mvs_rf, y_train = y_train, newx = X_test, newy = y_test)

# Repeated views MVS

# Create a new predictor matrix Xrep with repeated views
Xrep_train <- cbind(X_train, X_train)
Xrep_test <- cbind(X_test, X_test)
view_ids_rep <- c(view_ids, (view_ids + 4))

# Specify type of base learner per view
type <- list(
  level1 = c(
    "StaPLR",
    "StaPLR",
    "StaPLR",
    "StaPLR",
    "RF",
    "RF",
    "RF",
  )
)

```

```

      "RF"
    ),
    meta = "StaPLR"
  )

# Fit MVS model
mvs_rep <- MVS(
  x = Xrep_train,
  y = y_train,
  views = view_ids_rep,
  type = type,
  progress = FALSE,
  family = "gaussian",
  relax = c(FALSE, TRUE)
)

# Compute results
res_mvs_repeated[[i]] <- extract_results(mvs_rep, y_train = y_train, newx = Xrep_test, newy = y_test)

# MVS residual models

# Specify type of base learner per view
type <- list(
  level1 = c(
    "StaPLR",
    "StaPLR",
    "StaPLR",
    "StaPLR",
    "RF",
    "RF",
    "RF",
    "RF"
  ),
  meta = "StaPLR"
)

# Fit model with offset
mvs_res_mod <- MVS(
  x = Xrep_train,
  y = y_train,
  views = view_ids_rep,
  type = type,

```

```

    progress = FALSE,
    family = "gaussian",
    offsets = list(c(1, 5), c(2, 6), c(3, 7), c(4, 8)),
    relax = c(FALSE, TRUE)
)

# Compute results
# res_mvs_resid[[i]] <- extract_results(mvs_res_mod, y_train = y_train, newx = Xrep_test, n
res_mvs_resid[[i]] <- extract_results_lambda1se(mvs_res_mod, y_train = y_train, newx = Xrep
}

```

## References

Deming, D (2009). Early Childhood Intervention and Life-Cycle Skill Development: Evidence from Head Start. *American Economic Journal: Applied Economics* 1 (3): 111–34.